

SQL Joins Exercise

Moshe Matababe

Practice INNER JOIN, LEFT JOIN, and FULL OUTER JOIN on a five-table retail dataset.

```
103
104 --03: Part A-INNER JOIN
105 --QUESTION 01: Show every user who has a subscription. Match users to subscriptions.
106 SELECT A.user_id,
107        B.user_id,
108        user_name,
109        subscription_id,
110        start_date
111 FROM joins.exercises.users AS A
112      INNER JOIN joins.exercises.subscriptions AS B
113      ON A.user_id = B.user_id;
114
```

Add parameter

	user_id	user_id	user_name	subscription_id	start_date
1	1	1	Nomvula	503	2026-03-10
2	2	2	David	502	2026-02-01
3	3	3	Anele	505	2026-04-05
4	1	1	Nomvula	501	2026-01-15

```
115
116 --QUESTION 02: Show every subscription with its matching plan name and monthly price
117 SELECT subscription_id,
118        user_id,
119        A.plan_id,
120        B.plan_id,
121        plan_name,
122        monthly_price
123 FROM joins.exercises.subscriptions AS A
124      INNER JOIN joins.exercises.plans AS B
125      ON A.plan_id = B.plan_id;
126
127
```

Add parameter

	subscription_id	user_id	plan_id	plan_id	plan_name	monthly_price
1	501	1	10	10	Basic	79
2	504	6	11	11	Standard	129
3	503	1	12	12	Premium	199
4	505	3	13	13	Family	249
5	502	2	11	11	Standard	129

```

127
128 --QUESTION 3: Show every viewing session that has a matching show. Include the show title ad genre.
129 SELECT session_id,
130        user_id,
131        show_title,
132        genre,
133        watch_minutes
134 FROM joins.exercises.viewing_sessions AS A
135 INNER JOIN joins.exercises.shows AS B
136 ON A.show_id = B.show_id;
137

```

Add parameter

Table ▼ +

	¹ ₃ session_id	¹ ₃ user_id	^A _C show_title	^A _C genre	¹ ₃ watch_minutes
1	904	7	Comedy Hour	Comedy	20
2	903	1	Crime Time	Drama	60
3	902	2	Tech Tales	Documentary	30
4	901	1	Comedy Hour	Comedy	45

```

137
138
139 --QUESTION 4: Shwo every viewing session with the user who watched it. Only show sessions with a matching user.
140 SELECT user_name,
141        country,
142        session_id,
143        show_id,
144        watch_minutes
145 FROM joins.exercises.users AS A
146 INNER JOIN joins.exercises.viewing_sessions AS B
147 ON A.user_id = B.user_id;
148

```

Add parameter

Table ▼ +

	^A _C user_name	^A _C country	¹ ₃ session_id	¹ ₃ show_id	¹ ₃ watch_minutes
1	Nomvula	Johannesburg	903	702	60
2	David	Cape Town	902	703	30
3	Anele	Durban	905	705	90
4	Nomvula	Johannesburg	901	701	45

```

148
149
150 --QUESTION 5: Show users along with their subscriptions, the plan name, and the price. Use only users who have both a subscription and a valid plan.
151 SELECT user_name,
152        country,
153        plan_name,
154        monthly_price,
155        start_date
156 FROM joins.exercises.users AS A
157 INNER JOIN joins.exercises.subscriptions AS B
158 ON A.user_id = B.user_id
159 INNER JOIN joins.exercises.plans AS C
160 ON B.plan_id = C.plan_id;
161

```

Add parameter

Table ▼ +

	^A _C user_name	^A _C country	^A _C plan_name	¹ ₃ monthly_price	 start_date
1	Nomvula	Johannesburg	Premium	199	2026-03-10
2	David	Cape Town	Standard	129	2026-02-01
3	Anele	Durban	Family	249	2026-04-05
4	Nomvula	Johannesburg	Basic	79	2026-01-15

```

162
163 --04 Part B-LEFT JOIN
164 --QUESTION 06: Show every user and any subscriptions they have. Users without subscriptions must still appear.
165 SELECT A.user_id,
166        B.user_id,
167        user_name,
168        subscription_id,
169        start_date
170 FROM joins.exercises.users AS A
171 LEFT JOIN joins.exercises.subscriptions AS B
172 ON A.user_id = B.user_id;

```

Add parameter

	¹ ₃ user_id	¹ ₃ user_id	^A _C user_name	¹ ₃ subscription_id	 start_date
1	1	1	Nomvula	503	2026-03-10
2	2	2	David	502	2026-02-01
3	3	3	Anele	505	2026-04-05
4	4	null	Kabelo	null	null
5	5	null	Lerato	null	null
6	1	1	Nomvula	501	2026-01-15

```

173
174
175 --QUESTION 07: Show every plan and the subscriptions on it. Plans with no subscriptions must still appear.
176 SELECT A.plan_id,
177        plan_name,
178        subscription_id,
179        user_id
180 FROM joins.exercises.plans AS A
181 LEFT JOIN joins.exercises.subscriptions AS B
182 ON A.plan_id = B.plan_id;

```

Add parameter

	¹ ₃ plan_id	^A _C plan_name	¹ ₃ subscription_id	¹ ₃ user_id
1	10	Basic	501	1
2	11	Standard	504	6
3	12	Premium	503	1
4	13	Family	505	3
5	14	Mobile	null	null
6	11	Standard	502	2

```

183
184
185 --QUESTION 8: Show every show and any viewing sessions on it. Shows that were never watched must still appear.
186 SELECT A.show_id,
187        show_title,
188        session_id,
189        watch_minutes
190 FROM joins.exercises.shows AS A
191 LEFT JOIN joins.exercises.viewing_sessions AS B
192 ON A.show_id = B.show_id;

```

Add parameter

	show_id	show_title	session_id	watch_minutes
1	701	Comedy Hour	904	20
2	702	Crime Time	903	60
3	703	Tech Tales	902	30
4	704	Cooking Lab	null	null
5	706	Wild Earth	null	null
6	701	Comedy Hour	901	45

```

194
195 --QUESTION 9: Show every viewing session and the user who watched it. Sessions referencing users that do not exist must still appear (with NULL user details).
196 SELECT session_id,
197        show_id,
198        watch_minutes,
199        user_id,
200        user_name
201 FROM joins.exercises.viewing_sessions AS A
202 LEFT JOIN joins.exercises.users AS B
203 ON A.user_id = B.user_id;
204

```

Add parameter

	session_id	show_id	watch_minutes	user_id	user_name
1	901	701	45	1	Nomvula
2	902	703	30	2	David
3	903	702	60	1	Nomvula
4	904	701	20	7	null
5	905	705	90	3	Anele

```

204
205
206 --QUESTION 10: Show every user, the plan they are on (if any), and the monthly price. Users without a subscription must still appear.
207 SELECT user_name,
208        country,
209        plan_name,
210        monthly_price
211 FROM joins.exercises.users AS A
212 LEFT JOIN joins.exercises.subscriptions AS B
213 ON A.user_id = B.user_id
214 LEFT JOIN joins.exercises.plans AS C
215 ON B.plan_id = C.plan_id;

```

Add parameter

	user_name	country	plan_name	monthly_price
1	Nomvula	Johannesburg	Premium	199
2	David	Cape Town	Standard	129
3	Anele	Durban	Family	249
4	Kabelo	Pretoria	null	null
5	Lerato	Port Elizabeth	null	null
6	Nomvula	Johannesburg	Basic	79

```

216
217 --05: Part C-FULL OUTER JOIN
218 --QUESTION 11: Show every user and every subscription, including users without subscriptions AND subscriptions referencing users that do not exist.
219 SELECT A.user_id,
220        B.user_id,
221        A.user_name,
222        B.subscription_id,
223        A.start_date
224 FROM joins.exercises.users AS A
225 FULL OUTER JOIN joins.exercises.subscriptions AS B
226 ON A.user_id = B.user_id;

```

Add parameter

Table ▼ +

	i_3 user_id	i_3 user_id	A_0 user_name	i_3 subscription_id	i_3 start_date
1		5	null	Lerato	null null
2		1	1	Nomvula	503 2026-03-10
3		3	3	Anele	505 2026-04-05
4		2	2	David	502 2026-02-01
5		4	null	Kabelo	null null
6		1	1	Nomvula	501 2026-01-15
7		null	6	null	504 2026-03-20

```

228
229 --QUESTION 12: Show every plan and every subscription, including plans without subscribers AND any subscription referencing a plan that does not exist.
230 SELECT A.plan_id,
231        B.plan_id,
232        A.plan_name,
233        B.subscription_id,
234        A.user_id
235 FROM joins.exercises.plans AS A
236 FULL OUTER JOIN joins.exercises.subscriptions AS B
237 ON A.plan_id = B.plan_id;
238
239

```

Add parameter

Table ▼ +

	i_3 plan_id	i_3 plan_id	A_0 plan_name	i_3 subscription_id	i_3 user_id
1		12	12	Premium	503 1
2		10	10	Basic	501 1
3		13	13	Family	505 3
4		14	null	Mobile	null null
5		11	11	Standard	504 6
6		11	11	Standard	502 2

```

240
241 --QUESTION 13: Show every show and every viewing session, including shows that were never watched AND sessions referencing shows that do not exist.
242 SELECT A.show_id,
243        B.show_id,
244        A.show_title,
245        B.session_id,
246        A.watch_minutes
247 FROM joins.exercises.shows AS A
248 FULL OUTER JOIN joins.exercises.viewing_sessions AS B
249 ON A.show_id = B.show_id;
250
251

```

Add parameter

Table ▼ +

	i_3 show_id	i_3 show_id	A_0 show_title	i_3 session_id	i_3 watch_minutes
1		704	null	Cooking Lab	null null
2		703	703	Tech Tales	902 30
3		701	701	Comedy Hour	904 20
4		702	702	Crime Time	903 60
5		706	null	Wild Earth	null null
6		701	701	Comedy Hour	901 45
7		null	705	null	905 90

```
250
251
252 --QUESTION 14: Show every user and every viewing session, including users with no sessions AND sessions referencing users who do not exist.
253 SELECT A.user_id,
254        B.user_id,
255        user_name,
256        session_id,
257        show_id,
258        watch_minutes
259 FROM joins.exercises.users AS A
260 FULL OUTER JOIN joins.exercises.viewing_sessions AS B
261 ON A.user_id = B.user_id;
```

Add parameter

Table +

	¹ ₂ user_id	² ₃ user_id	⁰ ₄ user_name	⁵ ₃ session_id	⁶ ₃ show_id	⁷ ₃ watch_minutes
1		1	1 Nomvula		901	701 45
2		1	1 Nomvula		903	702 60
3		3	3 Anele		905	705 90
4		2	2 David		902	703 30
5	null		7 null		904	701 20
6		4	4 Kabelo	null	null	null null
7		5	5 Lerato	null	null	null null

```
264
265
266 --QUESTION 15: Show every user, every subscription, and every plan in one query-using FULL OUTER JOIN throughout. This is the hardest question-get all gaps visible at once.
267 SELECT A.user_id,
268        B.user_id,
269        user_name,
270        subscription_id,
271        B.plan_id,
272        C.plan_id,
273        plan_name
274 FROM joins.exercises.users AS A
275 FULL OUTER JOIN joins.exercises.subscriptions AS B
276 ON A.user_id = B.user_id
277 FULL OUTER JOIN joins.exercises.plans AS C
278 ON B.plan_id = C.plan_id;
```

Add parameter

Table +

	¹ ₂ user_id	² ₃ user_id	⁰ ₄ user_name	⁵ ₃ subscription_id	⁶ ₃ plan_id	⁷ ₃ plan_id	⁸ ₃ plan_name
1		1	1 Nomvula		503	12	12 Premium
2		1	1 Nomvula		501	10	10 Basic
3		3	3 Anele		505	13	13 Family
4	5	null	5 Lerato		null	null	null null
5		4	4 Kabelo	null	null	null	null null
6		2	2 David		502	11	11 Standard
7	null		6 null		504	11	11 Standard
8	null	null	null	null	null	14	14 Mobile